

DATABASE MANAGEMENT SYSTEMS
PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Java / Python

PROJECT TITLE

**DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT
SYSTEM FOR SALON AND SPA APPOINTMENT MANAGEMENT
SYSTEM
(PROJECT 66)**

Program	B.Tech – Computer Science and Engineering (AI & ML)
Section	AIML Rhinos
Faculty	Faculty name
Date of Submission	DD October 2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102069	Devyani Pawar	Mobile	University email

ACKNOWLEDGEMENT

I express my sincere gratitude to **Woxsen University, School of Technology**, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the **Database Management Systems (DBMS)** course.

I thank my course faculty, **[Faculty name]**, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled “**Design and Implementation of a Database Management System for Salon and Spa Appointment Management System**”, submitted as part of the Project-Based Learning activity for the **Database Management Systems (DBMS)** course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Signature

Date :	Name:	Devyani Pawar
Place : Hyderabad	Roll Number:	25WU0102069
	Program:	B.Tech CSE (AI & ML)

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	10
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	11
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	12
2. LOGICAL DATABASE DESIGN	13
2.1 Relational Schema	13
2.2 Keys and Integrity Constraints	14
2.3 Functional Dependencies	15
2.4 Normalization up to Third Normal Form (3NF)	16
2.5 Data Dictionary	17
2.6 Implementation of Logical Design: DDL and Sample Data	20
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	23
3.1 Application Overview	23
3.2 Development Environment and Database Connectivity	23
3.3 Application Modules and Database Mapping	24
3.4 CRUD Operations	24
3.5 Business Rules, Validation and Error Handling	24
3.6 Reports and Database Operations	25
3.7 Application Screens	26
3.8 Testing and Results	31
4. CONCLUSION	33
4.1 Limitations and Future Enhancements	33
5. REFERENCES	34
6. APPENDICES	35

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	11
Table 4	Entity and Relationship Summary	12
Table 5	Relational Schema	13
Table 6	Keys and Integrity Constraints	14
Table 7	Normalization Status up to 3NF	16
Table 8	Data Dictionary	17
Table 9	Database Implementation Environment	21
Table 10	Sample Data Summary	22
Table 11	Representative Records from v_appointment_details	22
Table 12	Application Development Environment	23
Table 13	Application Module-to-Database Mapping	24
Table 14	CRUD Operations	24
Table 15	Business Rules, Validation and Error Handling	25
Table 16	Reports and Database Operations	25
Table 17	Test Cases	31

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	Relational Schema Diagram (foreign keys point to referenced primary keys)	14
Figure 3	Dashboard with daily schedule and key figures	26
Figure 4	Customer registration rejecting an invalid phone number and email	26
Figure 5	Booking screen offering only skilled, on-shift and free staff	27
Figure 6	Booking rejected by the database for a skill mismatch and rolled back	27
Figure 7	Manage Appointments – appointment marked completed	28
Figure 8	Reschedule rejected for an overlapping staff appointment	28
Figure 9	Package session redeemed at no charge during booking	29
Figure 10	Bill preview with Gold membership discount and GST	29
Figure 11	Payment above the outstanding balance rejected	30
Figure 12	Staff utilization report	30
Figure 13	Monthly revenue report – billed vs collected	31
Figure 14	Full-page Entity-Relationship Diagram	36
Figure 15	Package balances report	37

LIST OF ACRONYMS

Acronym	Expansion
1NF / 2NF / 3NF	First / Second / Third Normal Form
BCNF	Boyce–Codd Normal Form
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER / ERD	Entity–Relationship / Entity–Relationship Diagram
FD	Functional Dependency
FK	Foreign Key
FR	Functional Requirement
GST	Goods and Services Tax
GUI	Graphical User Interface
INR	Indian Rupee
PBL	Project-Based Learning
PK	Primary Key
SQL	Structured Query Language
TC	Test Case
UPI	Unified Payments Interface

ABSTRACT

Salons and spas still commonly run their day on a paper appointment book, which cannot detect when a stylist is double-booked, when a service is given to someone without the right skill, how many prepaid package sessions a client has left, how much stock was consumed, or which bills are only partly paid. The objective of this project is to design and implement a relational database, with a small application on top of it, that manages the complete salon workflow from booking to payment while enforcing these rules automatically. The conceptual design identifies 14 entities and 16 relationships, which were mapped to 18 relations and normalized to Third Normal Form. The database was implemented in MySQL 8 with 85 declarative constraints, five triggers backed by a validation procedure, seven reporting views and seven secondary indexes, and populated with realistic data for 603 appointments. A Python application built with Streamlit provides customer registration, skilled-staff booking, package redemption, product issue, billing with membership discounts and GST, partial payments and six management reports. Testing showed that every invalid operation, including overlapping bookings, skill mismatches and over-payments, is rejected by the database itself, giving a dependable and practical system for small salons.

KEYWORDS

Keywords: Relational Database, ER Modelling, Normalization (3NF), MySQL Triggers, Python Streamlit, Salon Appointment Scheduling

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

A typical neighbourhood salon and spa offers 20 or more services across hair, skin, nails, massage, makeup and grooming, performed by staff who each hold only some of the required skills and work different weekly shifts. Customers buy prepaid packages and annual memberships, products such as hair colour and facial kits are consumed during services, and bills are often settled in more than one payment. In most small salons all of this is still recorded in a paper appointment book, separate package cards and a cash register.

This manual process has clear failures. The book cannot see that a stylist is already busy, so two clients are booked into overlapping slots; it does not know who is trained for which service, so a barber can be booked for a facial; package sessions are over-used or disputed; product consumption is never linked to services; and partly paid bills are forgotten. The receptionist, the salon manager, the service staff and ultimately the customer are all affected. A database-driven application is needed that stores this information once, enforces the salon's business rules at the moment of entry and produces reliable reports on appointments, staff utilization, service popularity, package balances, product use and revenue.

1.2 Objectives

- **Design** a normalized (3NF) relational database that covers all twelve required areas: customers, services, categories, staff, skills, schedules, appointments, packages, memberships, product usage, bills and payments.
- **Implement** the five core business rules inside MySQL – no overlapping staff appointments, skill match, package-session limits, valid appointment times and payment limits – so that no client can bypass them.
- **Populate** the database with realistic sample data (40 customers, 10 staff, 600+ appointments) and write at least 20 SQL queries and 6 views for the required reports.
- **Develop** a Python application that performs registration, booking with skilled-staff assignment, package use, product issue, billing, payment and reporting on the live database.

- **Validate** the system with at least 15 positive and negative test cases and record a five-minute demonstration.

1.3 Scope and Assumptions

Table 1 defines what the system covers and the assumptions made about the salon.

Table 1. Scope and Assumptions

Item	Description
In Scope	Customer registration and search; service categories and services; staff, skills with proficiency and weekly schedules; multi-service appointments with staff assignment; rescheduling, cancellation, completion and no-show; packages and package redemption; memberships and discounts; product stock and product issue; bills with discount and GST; full and partial payments; six management reports.
Out of Scope / Exclusions	Online self-booking portal, SMS/WhatsApp reminders, payroll and commissions, supplier purchase orders, multi-branch operation, user login and role-based access control.
Assumptions	Single branch open 09:00–21:00 every day; prices in INR; GST at 18% on the discounted amount; one bill per visit; a customer can hold at most one active membership; package sessions are redeemable only by the buyer before expiry; time slots in 15-minute steps.

1.4 Proposed Solution Strategy

The solution follows the standard database design life-cycle – requirements, conceptual ER design, logical relational design, normalization and physical implementation [1]. Business rules that involve only one row are declared as CHECK, UNIQUE, NOT NULL and FOREIGN KEY constraints; rules that need other rows (overlaps, skills, package balances, payment totals) are enforced by BEFORE INSERT/UPDATE triggers, so the database itself rejects invalid data. A Python application then provides the screens used at the front desk. Table 2 summarises the strategy.

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Receptionist (registration, booking, billing, payments); Salon manager (staff, skills, schedules, packages, stock, reports); Stylist / therapist (performs services, issues products); Customer (indirect user).
Major Application Modules	Dashboard, Customers, Book Appointment, Manage Appointments, Packages & Memberships, Products & Stock, Billing & Payments, Staff & Services, Reports.
Proposed Workflow	Register customer → book one or more services, choosing only skilled, on-shift and free staff (or redeem a package session) → complete the visit and issue products → generate the bill with membership discount and GST → record one or more payments → review reports.
Important Business Rules	No overlapping staff appointments; staff must hold the skill a service requires; slot = service duration and inside the staff shift and 09:00–21:00; package redemptions within sessions and validity, only by the owner; total payments ≤ bill total; bill only completed visits; stock never negative.

Expected Reports / Outputs	Daily appointment sheet and appointment statistics, staff utilization, service popularity and category revenue, package balances, product consumption with re-order alerts, monthly billed vs collected revenue and outstanding dues.
----------------------------	---

1.5 Functional Requirements

The twelve functional requirements in Table 3 were derived from the problem statement and the project brief.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Register, search, update and delete customers with input validation	High
FR-02	Maintain service categories, services, durations and prices	High
FR-03	Maintain staff, staff skills with proficiency, and weekly schedules	High
FR-04	Book an appointment containing one or more services and time slots	High
FR-05	Assign only staff who have the required skill, are on shift and are free	High
FR-06	Reschedule, cancel, complete or mark an appointment as no-show	High
FR-07	Sell packages and redeem sessions within limits and validity	High
FR-08	Enrol customers in membership plans and apply plan discounts on bills	Medium
FR-09	Issue products against performed services, update stock, flag re-order	Medium
FR-10	Generate a bill (subtotal, discount, GST, total) for a completed visit	High
FR-11	Record full or partial payments without exceeding the bill total	High
FR-12	Produce reports: appointments, staff utilization, service popularity, package balances, product use and revenue	High

1.6 Entity-Relationship Diagram (ERD)

The conceptual model was drawn using Chen-style ER notation [2]. Figure 1 shows the 14 entities with their key attributes and the 16 relationships with cardinalities. Weak entities (STAFF_SCHEDULE, APPOINTMENT_SERVICE and PAYMENT) have a double border and total participation is shown by a double line. Many-to-many relationships carry their own attributes – for example HAS_SKILL carries proficiency_level and CONTAINS carries sessions_included. A full-page version is given in Appendix C.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

The ER model was mapped to relations using the standard rules of the relational model [3]: each strong entity became a table with a surrogate primary key, weak entities received the owner's key as a foreign key, 1:N relationships became foreign keys on the N side, and every M:N relationship became its own table (staff_skill, package_item, customer_membership, product_usage). Table 5 lists the 18 relations and Figure 2 shows how they are linked.

Table 5. Relational Schema

S.No.	Table Name	Attributes (PK / FK)
1	service_category	category_id (PK), category_name, description
2	skill	skill_id (PK), skill_name
3	service	service_id (PK), category_id (FK), required_skill_id (FK), service_name, duration_min, price, is_active
4	customer	customer_id (PK), first_name, last_name, phone, email, gender, date_of_birth, registered_on
5	staff	staff_id (PK), full_name, phone, email, designation, hire_date, status
6	staff_skill	staff_id (PK, FK), skill_id (PK, FK), proficiency_level
7	staff_schedule	schedule_id (PK), staff_id (FK), day_of_week, shift_start, shift_end
8	membership_plan	plan_id (PK), plan_name, annual_fee, discount_pct
9	customer_membership	membership_id (PK), customer_id (FK), plan_id (FK), start_date, end_date
10	package	package_id (PK), package_name, price, validity_days, is_active
11	package_item	package_id (PK, FK), service_id (PK, FK), sessions_included
12	customer_package	customer_package_id (PK), customer_id (FK), package_id (FK), purchase_date, expiry_date, amount_paid, status
13	appointment	appointment_id (PK), customer_id (FK), appointment_date, status, booked_at, notes
14	appointment_service	appt_service_id (PK), appointment_id (FK), service_id (FK), staff_id (FK), start_time, end_time, price_charged, customer_package_id (FK)
15	product	product_id (PK), product_name, brand, unit, unit_cost, stock_qty, reorder_level
16	product_usage	usage_id (PK), appt_service_id (FK), product_id (FK), quantity_used, issued_at
17	bill	bill_id (PK), appointment_id (FK, UNIQUE), bill_date, subtotal, discount_amount, tax_amount, total_amount (generated)
18	payment	payment_id (PK), bill_id (FK), amount, method, paid_at, reference_no

Relational Schema - salon_spa db (18 tables)

PK = primary key · FK = foreign key · arrows point from FK to the referenced PK

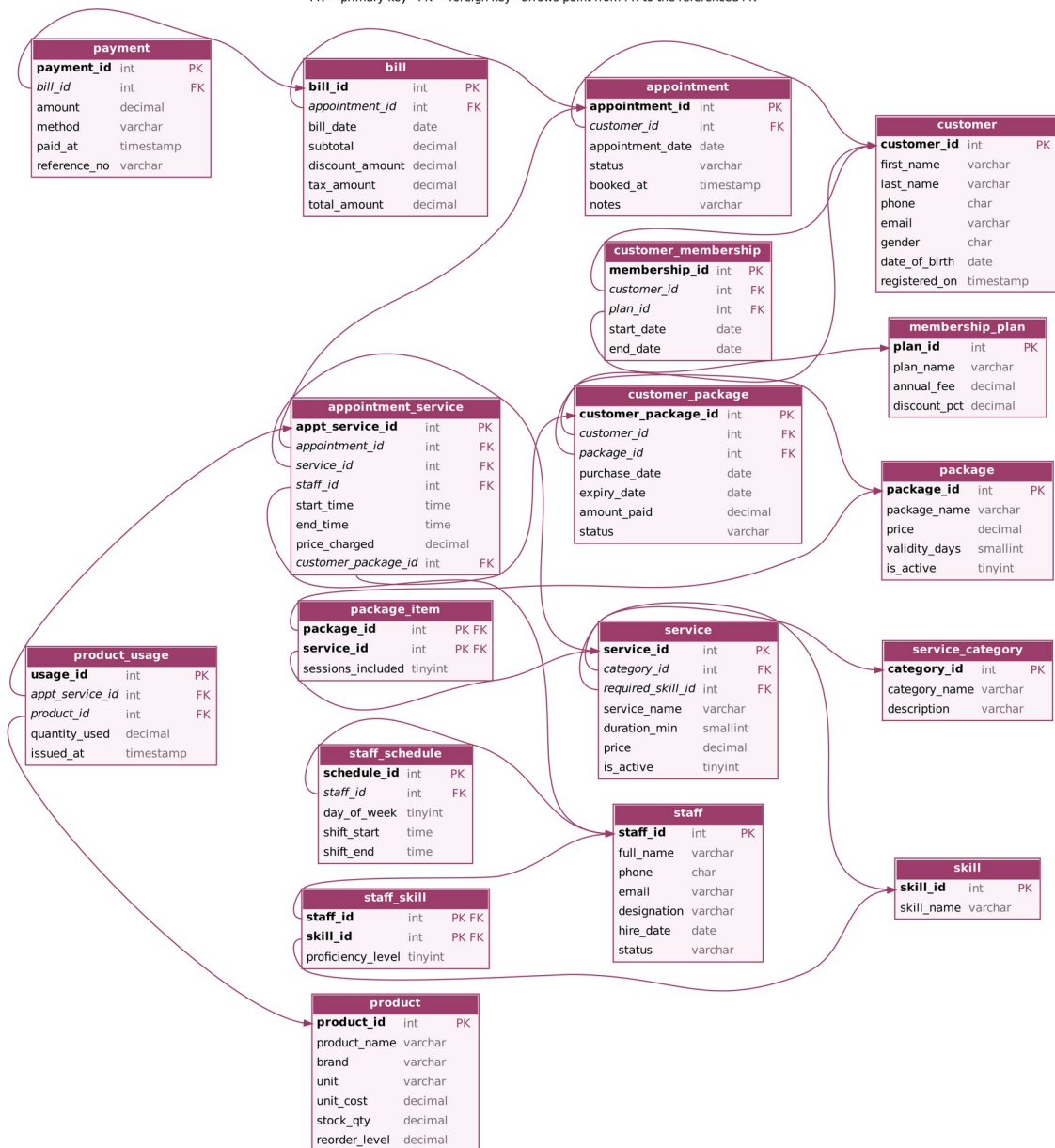


Figure 2. Relational Schema Diagram (foreign keys point to referenced primary keys)

2.2 Keys and Integrity Constraints

Table 6 lists the keys and constraints of every table. In total the schema declares 18 primary keys, 20 foreign keys, 12 UNIQUE constraints, 35 CHECK constraints, 17 DEFAULT values and 88 NOT NULL columns. Foreign keys use ON DELETE RESTRICT for history that must not be lost (a customer with appointments cannot be deleted), CASCADE for dependent detail rows (deleting a staff member removes his or her schedule and skills) and SET NULL for the optional package link.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
------------	------------------	--------------	------------------------

service_category	PK; UNIQUE	category_id; category_name	Each category listed once
skill	PK; UNIQUE	skill_id; skill_name	Each skill listed once
service	PK; FK; UNIQUE	service_id; category_id, required_skill_id; service_name	Service belongs to a category and needs a skill
service	CHECK; DEFAULT	duration_min 10–300, price > 0; is_active = 1	Realistic duration and price
customer	PK; UNIQUE; NOT NULL	customer_id; phone, email; first_name, last_name, phone	No duplicate customers
customer	CHECK; DEFAULT	phone ^[6-9][0-9]{9}\$, email pattern, gender F/M/O; registered_on = now	Valid Indian mobile number
staff	PK; UNIQUE; CHECK; DEFAULT	staff_id; phone, email; status in (ACTIVE, ON_LEAVE, INACTIVE); status = ACTIVE	Only active staff can be booked
staff_skill	PK; FK; CHECK; DEFAULT	(staff_id, skill_id); → staff CASCADE, → skill; level 1–5; 3	Skill match source
staff_schedule	PK; FK; UNIQUE; CHECK	schedule_id; staff_id; (staff_id, day_of_week); day 1–7, end > start, 09:00–21:00	One shift per staff per weekday
membership_plan	PK; UNIQUE; CHECK	plan_id; plan_name; fee ≥ 0, discount 0–50%	Bounded discounts
customer_membership	PK; FK; CHECK	membership_id; customer_id, plan_id; end_date > start_date	Valid membership period
package	PK; UNIQUE; CHECK	package_id; package_name; price > 0, validity 7–730 days	Valid packages
package_item	PK; FK; CHECK	(package_id, service_id); sessions 1–50	Package-session limit source
customer_package	PK; FK; CHECK; DEFAULT	customer_package_id; customer_id, package_id; expiry > purchase, status list; ACTIVE	Package validity
appointment	PK; FK; CHECK; DEFAULT	appointment_id; customer_id (RESTRICT); status list; BOOKED, booked_at = now	Visit lifecycle
appointment_service	PK; FK; CHECK	appt_service_id; appointment (CASCADE), service, staff, customer_package (SET NULL); end > start, 09:00–21:00, price ≥ 0	Valid appointment times
appointment_service	TRIGGER	trg_appt_service_bi / _bu → sp_validate_appt_service	No overlap, skill match, shift, duration, package limit
product	PK; UNIQUE; CHECK; DEFAULT	product_id; product_name; cost ≥ 0, stock ≥ 0, unit ml/g/pcs; unit = ml, stock = 0	Stock never negative
product_usage	PK; FK; CHECK; TRIGGER	usage_id; appt_service_id, product_id; quantity > 0; trg_product_usage_bi	Issue only available stock
bill	PK; FK + UNIQUE; CHECK; TRIGGER	bill_id; appointment_id; discount ≤ subtotal, tax ≥ 0; trg_bill_bi	One bill per completed visit
payment	PK; FK; CHECK; DEFAULT; TRIGGER	payment_id; bill_id; amount > 0, method list; UPI, paid_at = now; trg_payment_bi	Payment limit

2.3 Functional Dependencies

The important functional dependencies are listed below. In every case the determinant is a candidate key of its relation, which is the condition for BCNF.

customer_id	-> first_name, last_name, phone, email, gender, date_of_birth, registered_on
phone	-> customer_id (alternate key)

category_id	-> category_name, description
skill_id	-> skill_name
service_id	-> category_id, required_skill_id, service_name, duration_min, price, is_active
staff_id	-> full_name, phone, email, designation, hire_date, status
{staff_id, skill_id}	-> proficiency_level
schedule_id	-> staff_id, day_of_week, shift_start, shift_end
{staff_id, day_of_week}	-> schedule_id, shift_start, shift_end (alternate key)
plan_id	-> plan_name, annual_fee, discount_pct
membership_id	-> customer_id, plan_id, start_date, end_date
package_id	-> package_name, price, validity_days, is_active
{package_id, service_id}	-> sessions_included
customer_package_id	-> customer_id, package_id, purchase_date, expiry_date, amount_paid, status
appointment_id	-> customer_id, appointment_date, status, booked_at, notes
appt_service_id	-> appointment_id, service_id, staff_id, start_time, end_time, price_charged, customer_package_id
product_id	-> product_name, brand, unit, unit_cost, stock_qty, reorder_level
usage_id	-> appt_service_id, product_id, quantity_used, issued_at
bill_id	-> appointment_id, bill_date, subtotal, discount_amount, tax_amount
appointment_id	-> bill_id (1:1, UNIQUE)
payment_id	-> bill_id, amount, method, paid_at, reference_no

2.4 Normalization up to Third Normal Form (3NF)

Normalization was carried out on the paper booking slip, which in unnormalized form holds the customer's details, the date, a repeating group of {service, staff, staff skills, time, price}, the package used, the bill and a repeating group of payments [4].

- **1NF:** repeating groups were removed – one row per service per visit (appointment_service), one row per payment (payment) and one row per staff skill (staff_skill); every attribute became atomic.
- **2NF:** partial dependencies on composite keys were removed – service_name, duration and price depend only on service_id, and customer details only on customer_id, so they moved to service and customer.
- **3NF:** transitive dependencies were removed – category_name depends on category_id, skill_name on skill_id and the membership discount on plan_id, so service_category, skill and membership_plan were split out.

Three attributes look derivable but are stored deliberately as historical snapshots: price_charged and end_time in appointment_service (service price and duration can change after booking) and expiry_date in customer_package (package validity can change after purchase). Over time these are not functionally determined by the current service or package, so they do not violate 3NF. bill.total_amount is a generated column, and amount paid, balance and payment status are not stored at all but computed in the view v_bill_summary. Table 7 gives the final status.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCN	Remarks / Decomposition
---------------	-----	-----	-----	-----	-------------------------

				F	
service_category, skill	✓	✓	✓	✓	Single-attribute PK; names are alternate keys
service	✓	✓	✓	✓	Category and skill names moved out (3NF)
customer, staff	✓	✓	✓	✓	Phone and email are alternate keys
staff_skill	✓	✓	✓	✓	From staff's repeating skills (1NF); composite PK
staff_schedule	✓	✓	✓	✓	UNIQUE(staff_id, day_of_week) candidate key
membership_plan, customer_membership	✓	✓	✓	✓	Discount depends on plan only (3NF split)
package, package_item	✓	✓	✓	✓	M:N decomposed; sessions depend on full key
customer_package	✓	✓	✓	✓	expiry_date frozen at purchase
appointment	✓	✓	✓	✓	Header of the booking slip
appointment_service	✓	✓	✓	✓	Repeating service group (1NF); price/end time snapshots
product, product_usage	✓	✓	✓	✓	Usage separated from stock master
bill, payment	✓	✓	✓	✓	Repeating payments split (1NF); total generated

2.5 Data Dictionary

Table 8 is the data dictionary for all 18 tables, generated from the implemented schema.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
service_category	category_id	INT	PK, AUTO_INCREMENT	Category identifier
service_category	category_name	VARCHAR(50)	UNIQUE, NOT NULL	Name of the category
service_category	description	VARCHAR(200)		Short description
skill	skill_id	INT	PK, AUTO_INCREMENT	Skill identifier
skill	skill_name	VARCHAR(60)	UNIQUE, NOT NULL	Name of the skill
service	service_id	INT	PK, AUTO_INCREMENT	Service identifier
service	category_id	INT	FK → service_category, NOT NULL	Category identifier
service	required_skill_id	INT	FK → skill, NOT NULL	Skill needed to perform the service
service	service_name	VARCHAR(80)	UNIQUE, NOT NULL	Service name
service	duration_min	SMALLINT	NOT NULL	Duration in minutes
service	price	DECIMAL(8,2)	NOT NULL	Price in INR
service	is_active	TINYINT(1)	NOT NULL, DEFAULT 1	1 = available for sale
customer	customer_id	INT	PK, AUTO_INCREMENT	Customer identifier
customer	first_name	VARCHAR(40)	NOT NULL	First name
customer	last_name	VARCHAR(40)	NOT NULL	Last name
customer	phone	CHAR(10)	UNIQUE, NOT NULL	10-digit mobile number

Table	Attribute	Data Type	Key / Constraint	Description
customer	email	VARCHAR(100)	UNIQUE	Email address
customer	gender	CHAR(1)		F / M / O
customer	date_of_birth	DATE		Date of birth
customer	registered_on	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Registration timestamp
staff	staff_id	INT	PK, AUTO_INCREMENT	Staff identifier
staff	full_name	VARCHAR(80)	NOT NULL	Full name
staff	phone	CHAR(10)	UNIQUE, NOT NULL	10-digit mobile number
staff	email	VARCHAR(100)	UNIQUE	Email address
staff	designation	VARCHAR(40)	NOT NULL	Job title
staff	hire_date	DATE	NOT NULL	Date of joining
staff	status	VARCHAR(10)	NOT NULL, DEFAULT ACTIVE	ACTIVE / ON_LEAVE / INACTIVE
staff_skill	staff_id	INT	PK, FK → staff	Staff identifier
staff_skill	skill_id	INT	PK, FK → skill	Skill identifier
staff_skill	proficiency_level	TINYINT	NOT NULL, DEFAULT 3	Skill level 1–5
staff_schedule	schedule_id	INT	PK, AUTO_INCREMENT	Schedule row identifier
staff_schedule	staff_id	INT	FK → staff, UNIQUE, NOT NULL	Staff identifier
staff_schedule	day_of_week	TINYINT	UNIQUE, NOT NULL	1 = Sunday ... 7 = Saturday
staff_schedule	shift_start	TIME	NOT NULL	Shift start time
staff_schedule	shift_end	TIME	NOT NULL	Shift end time
membership_plan	plan_id	INT	PK, AUTO_INCREMENT	Plan identifier
membership_plan	plan_name	VARCHAR(30)	UNIQUE, NOT NULL	Silver / Gold / Platinum
membership_plan	annual_fee	DECIMAL(8,2)	NOT NULL	Fee per year (INR)
membership_plan	discount_pct	DECIMAL(5,2)	NOT NULL, DEFAULT 0.00	Discount on bills (%)
customer_membership	membership_id	INT	PK, AUTO_INCREMENT	Membership identifier
customer_membership	customer_id	INT	FK → customer, NOT NULL	Customer identifier
customer_membership	plan_id	INT	FK → membership_plan, NOT NULL	Plan identifier
customer_membership	start_date	DATE	NOT NULL	Membership start
customer_membership	end_date	DATE	NOT NULL	Membership end
package	package_id	INT	PK, AUTO_INCREMENT	Package identifier

Table	Attribute	Data Type	Key / Constraint	Description
package	package_name	VARCHAR(60)	UNIQUE, NOT NULL	Package name
package	price	DECIMAL(9,2)	NOT NULL	Package price (INR)
package	validity_days	SMALLINT	NOT NULL	Validity after purchase
package	is_active	TINYINT(1)	NOT NULL, DEFAULT 1	1 = available for sale
package_item	package_id	INT	PK, FK → package	Package identifier
package_item	service_id	INT	PK, FK → service	Service identifier
package_item	sessions_included	TINYINT	NOT NULL	Sessions of the service in the package
customer_package	customer_package_id	INT	PK, AUTO_INCREMENT	Purchased package identifier
customer_package	customer_id	INT	FK → customer, NOT NULL	Customer identifier
customer_package	package_id	INT	FK → package, NOT NULL	Package identifier
customer_package	purchase_date	DATE	NOT NULL	Date of purchase
customer_package	expiry_date	DATE	NOT NULL	Last date to redeem
customer_package	amount_paid	DECIMAL(9,2)	NOT NULL	Price paid (INR)
customer_package	status	VARCHAR(10)	NOT NULL, DEFAULT ACTIVE	ACTIVE / EXHAUSTED / EXPIRED / CANCELLED
appointment	appointment_id	INT	PK, AUTO_INCREMENT	Visit identifier
appointment	customer_id	INT	FK → customer, NOT NULL	Customer identifier
appointment	appointment_date	DATE	NOT NULL	Date of visit
appointment	status	VARCHAR(10)	NOT NULL, DEFAULT BOOKED	BOOKED / COMPLETED / CANCELLED / NO_SHOW
appointment	booked_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	When the booking was made
appointment	notes	VARCHAR(200)		Optional remarks
appointment_service	appt_service_id	INT	PK, AUTO_INCREMENT	Booked service line identifier
appointment_service	appointment_id	INT	FK → appointment, NOT NULL	Visit identifier
appointment_service	service_id	INT	FK → service, NOT NULL	Service identifier
appointment_service	staff_id	INT	FK → staff, NOT NULL	Staff identifier
appointment_service	start_time	TIME	NOT NULL	Slot start
appointment_service	end_time	TIME	NOT NULL	Slot end
appointment_service	price_charged	DECIMAL(8,2)	NOT NULL	Price at booking (0 if from package)
appointment_service	customer_package_id	INT	FK → customer_package	Purchased package identifier

Table	Attribute	Data Type	Key / Constraint	Description
product	product_id	INT	PK, AUTO_INCREMENT	Product identifier
product	product_name	VARCHAR(80)	UNIQUE, NOT NULL	Product name
product	brand	VARCHAR(40)		Brand
product	unit	VARCHAR(10)	NOT NULL, DEFAULT ml	ml / g / pcs
product	unit_cost	DECIMAL(8,2)	NOT NULL	Cost per unit (INR)
product	stock_qty	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	Quantity in stock
product	reorder_level	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	Re-order threshold
product_usage	usage_id	INT	PK, AUTO_INCREMENT	Usage identifier
product_usage	appt_service_id	INT	FK → appointment_service, NOT NULL	Booked service line identifier
product_usage	product_id	INT	FK → product, NOT NULL	Product identifier
product_usage	quantity_used	DECIMAL(8,2)	NOT NULL	Quantity issued
product_usage	issued_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Issue timestamp
bill	bill_id	INT	PK, AUTO_INCREMENT	Bill identifier
bill	appointment_id	INT	FK → appointment, UNIQUE, NOT NULL	Visit identifier
bill	bill_date	DATE	NOT NULL	Bill date
bill	subtotal	DECIMAL(10,2)	NOT NULL	Sum of service prices
bill	discount_amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	Membership discount
bill	tax_amount	DECIMAL(10,2)	NOT NULL, DEFAULT 0.00	GST (18%)
bill	total_amount	DECIMAL(10,2)	GENERATED	subtotal – discount + tax (generated)
payment	payment_id	INT	PK, AUTO_INCREMENT	Payment identifier
payment	bill_id	INT	FK → bill, NOT NULL	Bill identifier
payment	amount	DECIMAL(10,2)	NOT NULL	Amount paid
payment	method	VARCHAR(10)	NOT NULL, DEFAULT UPI	CASH / CARD / UPI / WALLET
payment	paid_at	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP	Payment timestamp
payment	reference_no	VARCHAR(40)		UPI / card reference

2.6 Implementation of Logical Design: DDL and Sample Data

The design was implemented in MySQL 8.0, whose reference manual was followed for CHECK constraints, generated columns, triggers and SIGNAL [5]. The scripts are split into

six files (Appendix A) and also run unchanged on MariaDB 10.11. Table 9 summarises the implementation environment.

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL Community Server (InnoDB storage engine)
DBMS Version	8.0 (tested on 8.0.46; also verified on MariaDB 10.11)
Number of Tables	18 tables, 7 views, 5 triggers, 1 stored procedure, 7 secondary indexes
Sample Data	3,245 rows: 40 customers, 10 staff, 22 services, 603 appointments with 892 service lines, 674 product issues, 427 bills, 419 payments (Aug–Oct 2026)

A representative CREATE TABLE statement, for the central appointment_service relation, is shown below. Every constraint is named so that error messages identify the rule that failed.

```
CREATE TABLE appointment_service (
  appt_service_id INT AUTO_INCREMENT PRIMARY KEY,
  appointment_id INT NOT NULL,
  service_id INT NOT NULL,
  staff_id INT NOT NULL,
  start_time TIME NOT NULL,
  end_time TIME NOT NULL,
  price_charged DECIMAL(8,2) NOT NULL,
  customer_package_id INT NULL,
  CONSTRAINT fk_as_appt FOREIGN KEY (appointment_id)
    REFERENCES appointment(appointment_id) ON UPDATE CASCADE ON DELETE CASCADE,
  CONSTRAINT fk_as_service FOREIGN KEY (service_id) REFERENCES service(service_id),
  CONSTRAINT fk_as_staff FOREIGN KEY (staff_id) REFERENCES staff(staff_id),
  CONSTRAINT fk_as_cpkg FOREIGN KEY (customer_package_id)
    REFERENCES customer_package(customer_package_id) ON DELETE SET NULL,
  CONSTRAINT chk_as_order CHECK (end_time > start_time),
  CONSTRAINT chk_as_hours CHECK (start_time >= '09:00:00'
    AND end_time <= '21:00:00'),
  CONSTRAINT chk_as_price CHECK (price_charged >= 0)
) ENGINE=InnoDB;
```

The rules that depend on other rows are checked in one stored procedure called by the BEFORE INSERT and BEFORE UPDATE triggers. The overlap check uses the interval-overlap condition (start A < end B and start B < end A):

```
SELECT COUNT(*) INTO v_cnt
FROM appointment_service s
JOIN appointment a ON a.appointment_id = s.appointment_id
WHERE s.staff_id = p_staff_id AND a.appointment_date = v_date
AND a.status NOT IN ('CANCELLED', 'NO_SHOW')
AND s.appt_service_id <> p_self_id -- ignore the row being updated
AND s.start_time < p_end AND p_start < s.end_time;
IF v_cnt > 0 THEN
  SIGNAL SQLSTATE '45000'
  SET MESSAGE_TEXT =
    'Staff conflict: staff member already has an overlapping appointment';
END IF;
```

Sample data was produced by a seeded Python generator that schedules every visit with the same rules, and it was loaded through the triggers, so every stored row is valid. Table 10 shows the number of rows per table and Table 11 shows representative records.

Table 10. Sample Data Summary

Table	Rows	Table	Rows
service_category	6	package	6
skill	10	package_item	11
service	22	customer_package	14
customer	40	appointment	603
staff	10	appointment_service	892
staff_skill	23	product	15
staff_schedule	60	product_usage	674
membership_plan	3	bill	427
customer_membership	10	payment	419

Table 11. Representative Records from v_appointment_details

Date	Start	Customer	Service	Staff	Price ₹	Status
2026-10-05	11:30	Aditi Agarwal	Classic Manicure	Lakshmi Prasanna	700	COMPLETED
2026-10-05	12:00	Sahithi Patel	Women's Haircut	Ananya Reddy	600	COMPLETED
2026-10-05	12:15	Aditi Agarwal	Spa Pedicure	Lakshmi Prasanna	1100	COMPLETED
2026-10-05	12:15	Varun Desai	Men's Haircut	Arjun Rao	350	COMPLETED
2026-10-05	12:45	Sahithi Patel	Blow Dry & Styling	Mohammed Irfan	450	COMPLETED
2026-10-05	12:45	Bhavana Chowdary	Hair Spa	Ananya Reddy	1500	COMPLETED

CHAPTER 3

APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application, branded “Glow & Grace Salon Manager” for the demonstration, is a browser-based Python program used at the front desk by the receptionist and by the salon manager. It has nine screens grouped as Front desk (Dashboard, Customers, Book Appointment, Manage Appointments), Operations (Packages & Memberships, Products & Stock, Billing & Payments, Staff & Services) and Insights (Reports). Every screen reads and writes the tables and views designed in Chapter 2 through a single data-access module; there is no static data in the program. Input format is validated in Python before the database is called, and all business rules are enforced again by the database, whose error messages are shown to the user unchanged.

3.2 Development Environment and Database Connectivity

Connectivity is handled by db.py. It opens a connection with autocommit disabled, runs every statement with %s parameters, and exposes a transaction() context manager that issues START TRANSACTION, COMMIT on success and ROLLBACK on any error. MySQL errors are translated into readable messages: SQLSTATE 45000 messages raised by triggers are passed through, duplicate keys, foreign-key violations and CHECK failures get their own wording. Connection settings are read from environment variables (SALON_DB_HOST, SALON_DB_USER, SALON_DB_PASSWORD, SALON_DB_NAME). Table 12 lists the environment.

Table 12. Application Development Environment

Item	Details
Programming Language	Python 3.11+
DBMS	MySQL 8.0
IDE / Framework / GUI Toolkit	VS Code; Streamlit (browser-based GUI written in Python) [7]; pandas for tabular data
Database Connector / Driver	mysql-connector-python (official Oracle driver) [6]; parameterised queries, explicit transactions
Operating Environment	Linux (Arch Linux) / Windows; runs locally at http://localhost:8501
Repository Link	<a href="https://github.com/<your-username>/salon-spa-dbms">https://github.com/<your-username>/salon-spa-dbms

3.3 Application Modules and Database Mapping

Table 13 maps each screen of the application to the tables and views it uses.

Table 13. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Dashboard	Daily snapshot	appointment, bill, v_appointment_details, v_bill_summary, v_product_usage	Read, aggregate
Customers	Registration and profile	customer, customer_membership	Create, search, update, delete
Book Appointment	Multi-service booking	appointment, appointment_service, service, staff_skill, staff_schedule, v_package_balance	Insert in one transaction
Manage Appointments	Lifecycle and reschedule	appointment, appointment_service	Search, update status/time/staff
Packages & Memberships	Sell and track	package, package_item, customer_package, membership_plan, customer_membership	Insert, read balances
Products & Stock	Issue and restock	product, product_usage	Insert usage, update stock
Billing & Payments	Bill and settle	bill, payment, v_bill_summary	Insert bill, insert payments
Staff & Services	Masters	staff, skill, staff_skill, staff_schedule, service	Read, update status/price, upsert skill
Reports	Six reports	seven views	Read, aggregate, CSV export

3.4 CRUD Operations

Table 14 shows where each create, read, update and delete operation is performed.

Table 14. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Register customer; book appointment; sell package; enrol membership; issue product; add product; generate bill; record payment	customer, appointment, appointment_service, customer_package, customer_membership, product_usage, product, bill, payment	Figures 4, 5, 9, 10
Read / Search	Customer search by name/phone/email; appointment search by date, status, customer; bill viewer; reports	all tables and views	Figures 3, 12, 13
Update	Edit customer; complete/cancel/no-show; reschedule time or staff; restock; change service price; staff status	customer, appointment, appointment_service, product, service, staff	Figures 7–8
Delete / Controlled Removal	Delete customer only after confirmation and only if no history (FK RESTRICT); remove a line from the booking cart before saving	customer	Test case TC-16

3.5 Business Rules, Validation and Error Handling

Validation works in two layers. The application checks input format before contacting the database (names, a 10-digit mobile number starting with 6–9, email pattern, non-future birth date, positive amounts, a reference number for UPI and card). It also prevents most rule

violations by design: time-slot lists only offer slots that end by 21:00, and the staff list shows only staff with the right skill who are on shift and free. The database is the second and final layer – triggers and constraints reject anything that still breaks a rule, for example when the staff filter is switched off or two receptionists book the same slot at once. All SQL uses bound parameters, which prevents SQL injection [8]. Destructive actions need a confirmation tick, and multi-statement operations run in a transaction that is rolled back on error. Table 15 lists the rules.

Table 15. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
No overlapping staff appointments	Staff list excludes busy staff; trigger sp_validate_appt_service (insert and update)	“Staff conflict: staff member already has an overlapping appointment” (Figure 8)
Skill match	Staff list joined with staff_skill; trigger re-check	“Skill mismatch: staff member does not have the skill required for this service” (Figure 6)
Valid appointment times	Slot list 09:00–21:00 in 15-min steps; CHECK chk_as_hours; trigger checks duration and staff shift	“Invalid time slot: this service takes 30 minutes” / “not on shift”
Package-session limit	Redeem option shown only with sessions left; trigger checks owner, validity, coverage, count	“Package session limit reached for this service”
Payment limit	Amount defaults to balance; trigger trg_payment_bi	“Payment exceeds balance: outstanding amount is Rs. ...” (Figure 11)
Stock and billing rules	trg_product_usage_bi, trg_bill_bi, CHECK stock_qty ≥ 0	“Insufficient stock: only 13.00 available”; “Bill can be generated only for a completed appointment”
Input format and uniqueness	Python regex checks; CHECK chk_customer_phone; UNIQUE phone/email	“Phone must be a 10-digit Indian mobile number...” (Figure 4); “Duplicate value: this phone already exists.”

3.6 Reports and Database Operations

The six required reports, the SQL concepts behind them and their output are listed in Table 16. Each report can be exported as CSV.

Table 16. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
Appointments	v_appointment_details (5-table join); COUNT(DISTINCT CASE ...) per day; correlated subquery for last visit	Daily totals by status, line chart, CSV
Staff utilization	v_staff_utilization: derived tables, CROSS JOIN, SUM of minutes ÷ shift minutes	Utilization % per staff (Figure 12)
Service popularity	v_service_popularity: LEFT JOIN with IN-subquery, GROUP BY; category revenue share	Bookings, package use and revenue per service
Package balances	v_package_balance: correlated subquery counting sessions used	Included / used / left per customer package
Product use	v_product_usage: SUM, CASE re-order flag; average material cost per service	Consumption cost and re-order alerts
Revenue	v_monthly_revenue, v_bill_summary: GROUP BY month, billed vs collected, outstanding	Monthly bar chart and dues (Figure 13)

3.7 Application Screens

Figures 3 to 13 show the main screens running on the sample database.

Dashboard

Today's front-desk snapshot, pulled live from the database.

Show schedule for

2026 / 10 / 08

Appointments on day

10

Billed this month

₹96,967.68

Outstanding dues

₹8,500.43

Products to reorder

1

Registered customers

41

Schedule • Thu, 08 Oct 2026

Start	End	Customer	Service	Staff	Status	Package
10:00	10:30	Karthik Patel	Men's Haircut	Kavya Sharma	BOOKED	No
11:30	11:45	Aditi Agarwal	Eyebrow Threading	Priya Naidu	BOOKED	No
11:45	12:45	Aditi Agarwal	Classic Facial	Fatima Begum	BOOKED	No
12:00	13:00	Pooja Patel	Swedish Massage (60 min)	Rohan Verma	BOOKED	No
13:00	13:30	Pooja Patel	Head & Shoulder Massage	Rohan Verma	BOOKED	No
13:45	14:45	Charitha Jain	Spa Pedicure	Sneha Kulkarni	BOOKED	No
15:15	15:45	Sai Teja Yadav	Men's Haircut	Arjun Rao	BOOKED	No
15:15	16:00	Bhavana Pillai	Classic Manicure	Sneha Kulkarni	BOOKED	No
15:45	16:45	Pooja Patel	Swedish Massage (60 min)	Rohan Verma	BOOKED	No
16:45	17:45	Pranav Rao	Swedish Massage (60 min)	Rohan Verma	BOOKED	No

Monthly revenue

Top 8 services

Figure 3. Dashboard with daily schedule and key figures

Customers

Register, search, update and remove customers.

+ Register Search & Edit Visit history

First name *

Last name *

Mobile number *

10 digits, starts with 6-9

Email

Gender

Female

Date of birth

yyyy / mm / dd

Register customer

Phone must be a 10-digit Indian mobile number starting with 6-9.

Email address is not valid.

Figure 4. Customer registration rejecting an invalid phone number and email

Book Appointment

Pick the customer, date and services. Only staff who have the required skill, are on shift and are free are offered. The database re-checks every rule when you confirm.

Search customer by name or phone

Meera

Appointment date

2026 / 10 / 08

Customer

Meera Iyer · 9876501234 (#43)

Add a service

Service

Skin & Facial · Classic Facial (60 min, ₹1,400.00)

Start time

05:00 PM

Required skill: Facial & Skin Care · ends at 06:00 PM

☒ Show only available skilled staff ⓘ

Assign staff

Priya Naidu — Skin Therapist · skill level 5/5

+ Add to booking

Booking summary

Figure 5. Booking screen offering only skilled, on-shift and free staff

Makeup · Party Makeup (75 min, ₹3,000.00)

07:00 PM

Required skill: Makeup Artistry · ends at 08:15 PM

☐ Show only available skilled staff ⓘ

Assign staff

Arjun Rao — Barber

+ Add to booking

Booking summary

07:00 PM – 08:15 PM	Party Makeup	Arjun Rao	₹3,000.00	✕
---------------------	--------------	-----------	-----------	---

Estimated amount (before discount & GST): ₹3,000.00

Notes (optional)

✔ Confirm booking

✕ Booking rejected and rolled back: Skill mismatch: staff member does not have the skill required for this service

Figure 6. Booking rejected by the database for a skill mismatch and rolled back

Manage Appointments

Search appointments, update their status, reschedule or reassign staff.

Marked completed. You can now raise the bill.

From To Status BOOKED ✕ COMPLETED ✕ ▼ Customer name / phone

1 appointment(s), 2 service line(s)

Appt	Date	Start	End	Customer	Phone	Service	Staff	Price ₹	Package	Status
606	2026-10-08	17:00	18:00	Meera Iyer	9876501234	Classic Facial	Priya Naidu	1400	No	COMPLETED
606	2026-10-08	18:00	18:15	Meera Iyer	9876501234	Eyebrow Threading	Priya Naidu	80	No	COMPLETED

Select appointment

#606 – Meera Iyer on 2026-10-08

Current status: COMPLETED

Figure 7. Manage Appointments – appointment marked completed

Select appointment

#487 – Meghana Yadav on 2026-10-10

Current status: BOOKED

✓ Mark completed

☐ Confirm cancellation

☐ Mark no-show

✕ Cancel appointment

Reschedule / reassign a service

Service line

Eyebrow Threading @ 16:30

Same-day changes only; to move to another date, cancel and book again.

New start

03:30 PM

Staff (skilled only)

Priya Naidu

Save reschedule

✕ Staff conflict: staff member already has an overlapping appointment

Figure 8. Reschedule rejected for an overlapping staff appointment

Add a service

Service

Nails · Classic Manicure (45 min, ₹700.00) ▼

Start time

07:00 PM ▼

Required skill: Manicure & Pedicure · ends at 07:45 PM

☒ Show only available skilled staff ⓘ

No skilled staff member is free for this slot. Try another time.

Booking summary

07:00 PM – 07:45 PM Classic Manicure Sneha Kulkarni 📅 package ✕

Estimated amount (before discount & GST): ₹0.00

Notes (optional)

Confirm booking

Figure 9. Package session redeemed at no charge during booking

Billing & Payments

Raise bills for completed appointments and record payments.

[Generate bill](#) [Record payment](#) [View bill](#)

Completed appointment without a bill

#605 – Pooja Mishra – 2026-10-08 ▼

Service	Staff	Package	Amount ₹
Women's Haircut	Mohammed Irfan	No	600

Subtotal

₹600.00

Discount (Gold 10%)

₹60.00

GST 18%

₹97.20

Total

₹637.20

Generate bill

Figure 10. Bill preview with Gold membership discount and GST

Billing & Payments

Raise bills for completed appointments and record payments.

[Generate bill](#) [Record payment](#) [View bill](#)

Bill with balance due

Bill #428 – Meera Iyer – balance ₹1,746.40

Bill total

₹1,746.40

Paid so far

₹0.00

Balance due

₹1,746.40

Amount ₹

5000.00

Method

☒ UPI ☐ CARD ☐ CASH ☐ WALLET

Transaction reference (UPI/card)

UPI4471029

Record payment

✗ Payment exceeds balance: outstanding amount is Rs. 1746.40

Figure 11. Payment above the outstanding balance rejected

Reports

Six management reports, each backed by a SQL view or aggregate query.

[Appointments](#) [Staff utilisation](#) [Service popularity](#) [Package balances](#) [Product use](#) [Revenue](#)

Utilisation = minutes of completed services ÷ scheduled shift minutes over the period.

Staff	Role	Services	Minutes	Revenue ₹	Utilisation %
Rohan Verma	Spa Therapist	119	6480	215400	20.9
Lakshmi Prasanna	Beautician	117	4920	90600	15.9
Ananya Reddy	Senior Stylist	58	3885	87750	12.5
Priya Naidu	Skin Therapist	103	3180	55060	10.3
Fatima Begum	Makeup Artist	53	3105	78600	10
Kavya Sharma	Colour Specialist	47	2820	60450	9.1
Sneha Kulkarni	Nail Technician	49	2790	51700	9
Mohammed Irfan	Stylist	51	2185	39250	7
Arjun Rao	Barber	41	1345	16250	4.3
Deepika Goud	Beautician	0	0	0	0

Download staff_utilisation.csv

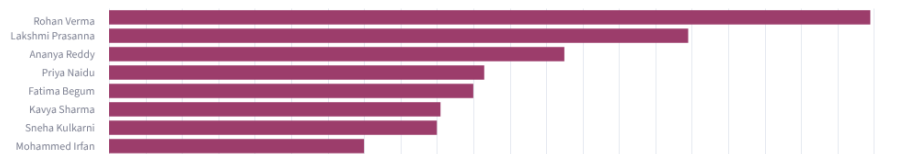


Figure 12. Staff utilization report

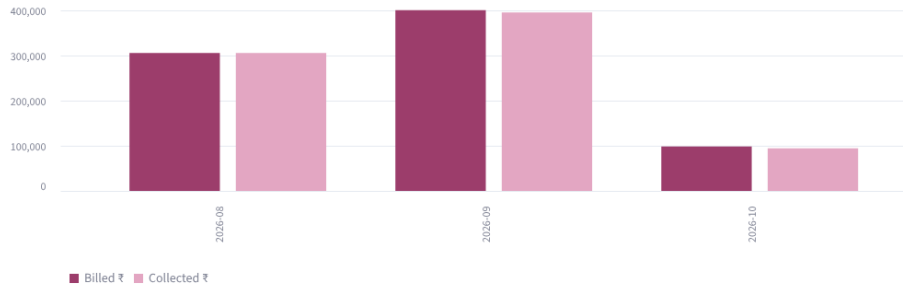
Reports

Six management reports, each backed by a SQL view or aggregate query.

📅 Appointments 🧑 Staff utilisation ⭐ Service popularity 📦 Package balances 🛒 Product use 💰 Revenue

Month	Bills	Gross ₹	Discounts ₹	GST ₹	Billed ₹	Collected ₹
2026-08	155	265680	6109.5	46722.69	306293.19	306293.19
2026-09	217	344730	4677.5	61209.45	401261.95	396335.45
2026-10	56	84650	994	15058.08	98714.08	94393.75

Download revenue.csv



Collections by payment method

Method	Payments	Amount ₹
--------	----------	----------

Figure 13. Monthly revenue report – billed vs collected

3.8 Testing and Results

Testing combined a database-level script (database/06_test_business_rules.sql, run in a transaction that is rolled back) and manual tests through the application. Table 17 lists the main positive and negative cases with the observed results.

Table 17. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Booking (DB)	Valid Swedish massage 13:00–14:00 for Rohan	Row inserted	1 row inserted	Pass
TC-02	Overlap rule	Second booking for Rohan 13:30–14:30	Rejected	Staff conflict error	Pass
TC-03	Overlap rule	Back-to-back booking 14:00–15:00	Accepted	Inserted	Pass
TC-04	Skill match	Bridal makeup assigned to the barber	Rejected	Skill mismatch error	Pass
TC-05	Valid times	30-min haircut booked for 45 min	Rejected	Invalid time slot error	Pass
TC-06	Valid times	Massage at 10:00, before the 12:00 shift	Rejected	Not on shift error	Pass
TC-07	Package limit	Redeem another customer's package	Rejected	Package does not belong error	Pass
TC-08	Package limit	Redeem 4th of 4, then a 5th manicure	4th at ₹0; 5th rejected	₹0.00, then limit error	Pass
TC-09	Billing rule	Bill a BOOKED appointment	Rejected	Completed-only error	Pass
TC-10	Payment limit	Pay balance + 100, then exact balance	First rejected, then PAID	Error, then status PAID	Pass
TC-11	Stock rule	Issue 500 gold facial kits	Rejected	Insufficient stock error	Pass

TC-12	Registration (app)	Phone 12345, email meera.iyer@gmail	Two validation errors	Both errors shown (Fig. 4)	Pass
TC-13	Booking (app)	Facial 17:00 + threading 18:00 for new customer	Booked with skilled staff	Confirmation shown	Pass
TC-14	Reschedule (app)	Move threading to 15:30 onto a facial slot	Rejected	Staff conflict (Fig. 8)	Pass
TC-15	Billing (app)	₹600 haircut for Gold member	₹60 off, GST ₹97.20, total ₹637.20	As expected (Fig. 10)	Pass
TC-16	Delete (app)	Delete customer who has appointments	Blocked by FK	“Cannot delete ...” message	Pass
TC-17	Uniqueness	Register a duplicate phone number	Rejected	Duplicate value message	Pass
TC-18	Check constraint	Discount larger than subtotal	Rejected	chk_bill_discount violated	Pass

All 18 test cases produced the expected result. Every invalid operation was stopped by the database even when the application's own filters were bypassed, and multi-statement bookings were rolled back completely on error, so no partial data was left behind. Valid edge cases – such as back-to-back appointments and redeeming the last package session – were accepted. The application therefore uses the designed database correctly and enforces all of the important business rules.

CHAPTER 4

CONCLUSION

This project replaced the error-prone paper appointment book of a salon and spa with a relational database and a Python application. Starting from the requirements, an ER model of 14 entities and 16 relationships was designed and converted into 18 relations in Third Normal Form. The MySQL implementation uses 85 declarative constraints, five triggers, seven views and seven indexes, so that overlapping staff bookings, skill mismatches, invalid times, package over-use and over-payment are rejected by the database itself. The Streamlit application supports the full workflow from registration to payment and produces the six required reports.

The main learning was that placing business rules inside the database, rather than only in the user interface, keeps the data correct no matter which program or person changes it, and that careful normalization made the reports simple to write as views.

4.1 Limitations and Future Enhancements

- There is no login yet; role-based access for receptionist, manager and staff can be added with a user table and permissions.
- The design assumes one branch; adding branch_id to staff, appointments and stock would support a chain of salons.
- Appointment reminders, a customer self-booking portal and online payment links are not implemented.
- Staff utilization uses scheduled shift time only; leave and holidays could be modelled for more accurate utilization.

REFERENCES

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [2] P. P. Chen, “The entity-relationship model—Toward a unified view of data,” *ACM Trans. Database Syst.*, vol. 1, no. 1, pp. 9–36, Mar. 1976, doi: 10.1145/320434.320440.
- [3] E. F. Codd, “A relational model of data for large shared data banks,” *Commun. ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970, doi: 10.1145/362384.362685.
- [4] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [5] Oracle Corporation, *MySQL 8.0 Reference Manual*. Accessed: Oct. 8, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>
- [6] Oracle Corporation, *MySQL Connector/Python Developer Guide*. Accessed: Oct. 8, 2026. [Online]. Available: <https://dev.mysql.com/doc/connector-python/en/>
- [7] Snowflake Inc., *Streamlit Documentation*. Accessed: Oct. 8, 2026. [Online]. Available: <https://docs.streamlit.io/>
- [8] OWASP Foundation, “SQL injection prevention cheat sheet,” *OWASP Cheat Sheet Series*. Accessed: Oct. 8, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete script is submitted in the repository folder database/ (<https://github.com/<your-username>/salon-spa-dbms>). Run the single file salon_spa_full.sql, or the parts in order:

- 01_schema.sql – database, 18 tables, constraints and indexes (DDL)
- 02_triggers.sql – validation procedure and five business-rule triggers
- 03_sample_data.sql – sample data (DML), generated by generate_sample_data.py
- 04_views.sql – seven reporting views
- 05_queries.sql – 20 retrieval queries: joins, subqueries, aggregation, views
- 06_test_business_rules.sql – 18 positive / negative rule tests (rolled back)

APPENDIX B - APPLICATION SOURCE CODE

Repository: <https://github.com/<your-username>/salon-spa-dbms>. The application is in the app/ folder: streamlit_app.py (entry point and navigation), db.py (connection, parameterised queries, transactions, error mapping), ui.py (validation and shared widgets) and views/ (one file per screen). Run: pip install -r requirements.txt, then streamlit run app/streamlit_app.py. The five-minute demonstration video link is in video/VIDEO_LINK.md.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

Output of the business-rule test script on MySQL 8.0 (abridged):

```
TC-01 valid booking                                rows_ok = 1
ERROR 1644 (45000): Staff conflict: staff member already has an overlapping appointment
ERROR 1644 (45000): Skill mismatch: staff member does not have the required skill
ERROR 1644 (45000): Invalid time slot: this service takes 30 minutes
ERROR 1644 (45000): Invalid time: staff member is not on shift for this slot
ERROR 1644 (45000): Package does not belong to this customer
TC-09 package redemption                            price_charged = 0.00
ERROR 1644 (45000): Package session limit reached for this service
ERROR 1644 (45000): Bill can be generated only for a completed appointment
ERROR 1644 (45000): Payment exceeds balance: outstanding amount is Rs. 1888.00
TC-13 settle bill                                  payment_status = PAID, balance = 0.00
ERROR 1644 (45000): Insufficient stock: only 17.00 available
ERROR 3819 (HY000): Check constraint 'chk_customer_phone' is violated.
ERROR 1062 (23000): Duplicate entry '9374481727' for key 'customer.phone'
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key ... fails
ERROR 3819 (HY000): Check constraint 'chk_bill_discount' is violated.
All tests finished - transaction rolled back
```

Figure 14 is the full-page ER diagram (rotated) and Figure 15 the package-balance report.

Salon & Spa Appointment Management System – ER Diagram

Underlined = primary key, italic = foreign key (U) = unique, double border = weak entity, double line = total participation

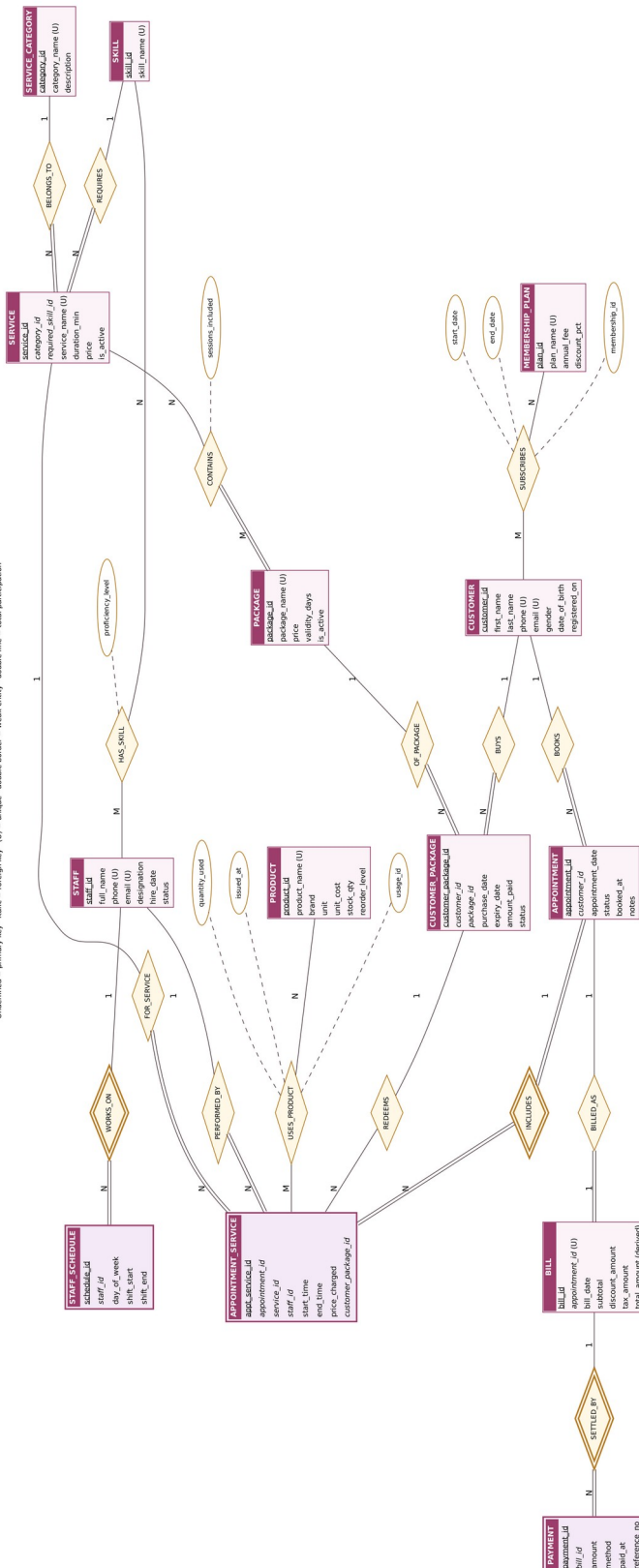


Figure 14. Full-page Entity-Relationship Diagram

Reports

Six management reports, each backed by a SQL view or aggregate query.

 Appointments  Staff utilisation  Service popularity  **Package balances**  Product use  Revenue

Customer	Package	Service	Included	Used	Left	Expires	Status
Sahithi Patel	Mani-Pedi Monthly	Spa Pedicure	4	3	1	2026-10-31	ACTIVE
Sahithi Patel	Mani-Pedi Monthly	Classic Manicure	4	3	1	2026-10-31	ACTIVE
Harika Mishra	Mani-Pedi Monthly	Spa Pedicure	4	1	3	2026-11-14	ACTIVE
Harika Mishra	Mani-Pedi Monthly	Classic Manicure	4	1	3	2026-11-14	ACTIVE
Ritu Reddy	Bridal Prep Package	Bridal Makeup	1	0	1	2026-11-19	ACTIVE
Ritu Reddy	Bridal Prep Package	Full Arms Waxing	2	0	2	2026-11-19	ACTIVE
Ritu Reddy	Bridal Prep Package	Eyebrow Threading	2	0	2	2026-11-19	ACTIVE
Ritu Reddy	Bridal Prep Package	Gold Facial	2	0	2	2026-11-19	ACTIVE
Ritu Gupta	Mani-Pedi Monthly	Spa Pedicure	4	1	3	2026-12-02	ACTIVE
Ritu Gupta	Mani-Pedi Monthly	Classic Manicure	4	1	3	2026-12-02	ACTIVE

 Download package_balances.csv

Figure 15. Package balances report